

Παράλληλη Επεξεργασία

2023 - 2024

Χαράλαμπος Γιαννέλης AM : 1093341

Γιώργος Σωτηρόπουλος AM : 1072541

Παναγιώτης Πέττας AM : 1093480

Αναστασία Καπελλάκη AM : 1072492

Πίνακας περιεχομένων

Εισαγωγή	3
<i>Variables Explanation</i>	<i>3</i>
<i>Rosenbrock Function.....</i>	<i>3</i>
<i>Problem Parameters</i>	<i>4</i>
<i>MDS Parameters.....</i>	<i>4</i>
<i>Points and Results.....</i>	<i>4</i>
<i>Best Point Information.....</i>	<i>5</i>
<i>Local Variables.....</i>	<i>5</i>
<i>Multistart Trials</i>	<i>5</i>
Ερώτημα 1 : OpenMP	6
Ερώτημα 2 : OpenMP Tasks	11
Ερώτημα 3 : MPI.....	15
<i>MPI enviroment</i>	<i>15</i>
<i>Work Distribution.....</i>	<i>15</i>
<i>MPI_Reduce</i>	<i>19</i>
<i>Print Results</i>	<i>20</i>
Αποτελέσματα.....	20
<i>Αρχιτεκτονική CPU</i>	<i>20</i>
<i>Μετρήσεις.....</i>	<i>21</i>
OpenMP	22
OpenMP Tasks.....	22
MPI	23
<i>Σύγκριση Μεθόδων</i>	<i>23</i>
Συμπεράσματα.....	24

Εισαγωγή

Παρακάτω θα αναλύσουμε την χρήση της κάθε μεταβλητής κρίσιμο κομμάτι για να μπορέσουμε να αξιολογήσουμε ποιες μεταβλητές θα πρέπει να προσέξουμε κατά την διαδικασία της παραλληλοποίησης.

Variables Explanation

- **MAXVARS**: Maximum αριθμός μεταβλητών (dimensions) τις οποίες μπορεί να διαχειριστεί ο κώδικας.
- **EPSMIN**: Minimum μέγεθος convergence δηλαδή βήματος σύγκλισης, η οποία εκφράζει την ακρίβεια της τελικής λύσης.
- **funevals**: global variable η οποία παρακολουθεί τον συνολικό αριθμό των Evaluation Function που πραγματοποιήθηκαν κατά τη διάρκεια επίλυσης του προβλήματος.

Rosenbrock Function

f(double *x, int n): Υπολογίζει την τιμή της συνάρτησης Rosenbrock για ένα n-διάστατο σημείο x.

- **x**: Το input το οποίο αντιπροσωπεύει το σημείο στον Simplex.
- **n**: Ο αριθμός των διαστάσεων (dimensions).
- **fv**: Η τιμή η οποία υπολογίζεται από την Rosenbrock function.
- **funevals++**: Αυξάνει τον global μετρητή ο οποίος μετρά των συνολικό αριθμό υπολογισμού της evaluation function.
- **usleep(1)**: Προσθέτει μια καθυστέρηση για κάθε αξιολόγηση συνάρτησης.

Problem Parameters

- **nvars**: Αριθμός διαστάσεων (dimensions) του προβλήματος.
- **ntrials**: Αριθμός τυχαίων σημείων εκκίνησης για τη Multistart μέθοδο.
- **lower - upper**: Arrays τα οποία καθορίζουν τα κάτω και άνω όρια του simplex για κάθε μεταβλητή.

MDS Parameters

- **eps**: κριτήριο σύγκλισης. Ο αλγόριθμος σταματά όταν το μέγεθος βήματος είναι μικρότερο από αυτή την τιμή.
- **maxfevals**: maximum αριθμός για function evaluations..
- **maxiter**: maximum αριθμός για iterations.
- **mu**: συντελεστής επέκτασης (Για την πράξη expansion του MDS).
- **theta**: συντελεστής Contraction (Για την πράξη constraction του MDS).
- **delta**: δηλώνει το βήμα για τα vertices στον πολύτοπο simplex.

Points and Results

- **startpt**: Array για την αποθήκευση του starting point για τον MDS.
- **endpt**: Array για την αποθήκευση του final point για τον MDS.
- **fx**: Τιμή της συνάρτησης Rosenbrock στο τέλος της εκτέλεσης.
- **nt - nf**: αριθμός iterations και αριθμός function evaluations οι οποίοι χρησιμοποιήθηκαν κατά την εκτέλεση του MDS.

Best Point Information

- **best_pt**: για την αποθήκευση των συντεταγμένων του καλύτερου σημείου που βρέθηκε κατά τη διαδικασία Multistart.
- **best_fx**: Τιμή συνάρτησης στο καλύτερο σημείο.
- **best_trial**: Index του βήματος που βρέθηκε το καλύτερο σημείο.
- **best_nt - best_nf**: Αριθμός επαναλήψεων και υπολογισμού Evaluation Function για την καλύτερη δοκιμή.

Local Variables

- **trial, i**: Μετρητές επαναλήψεων.
- **t0, t1**: Μετρητές χρόνου.

Multistart Trials

Ο παρακάτω κώδικας χρησιμοποιείται για την εκτέλεση πολλαπλών δοκιμών του αλγορίθμου MDS από διαφορετικά σημεία εκκίνησης και την επιλογή της καλύτερης λύσης που βρέθηκε μεταξύ αυτών των δοκιμών. Το for loop επαναλαμβάνει τον αριθμό δοκιμών που καθορίζεται από το ntrials. Κάθε επανάληψη αντιπροσωπεύει μια νέα δοκιμή με διαφορετικό σημείο εκκίνησης για τον αλγόριθμο MDS.

```
83   for (trial = 0; trial < ntrials; trial++) {
84
85       srand48(trial);
86
87       /* starting guess for rosenbrock test function, search space in [-2, 2) */
88
89       for (i = 0; i < nvars; i++) {
90           startpt[i] = lower[i] + (upper[i]-lower[i])*drand48();
91       }
92
93       int term = -1;
94
95       mds(startpt, endpt, nvars, &fx, eps, maxfevals, maxiter, mu, theta, delta,
96          &nt, &nf, lower, upper, &term);
97
98       /* keep the best solution */
99
100      if (fx < best_fx) {
101          best_trial = trial;
102          best_nt = nt;
103          best_nf = nf;
104          best_fx = fx;
105          for (i = 0; i < nvars; i++)
106              best_pt[i] = endpt[i];
107      }
108  }
```

srand48 : σκοπός του srand48 είναι να τροφοδοτεί τη γεννήτρια τυχαίων αριθμών που χρησιμοποιείται από το drand48, με τον τρέχοντα αριθμό του trial. Η χρήση του trial εξασφαλίζει ότι σε κάθε επανάληψη θα έχουμε διαφορετικό σημείο εκκίνησης. Είναι σημαντικό να τονίσουμε ότι η χρήση του ίδιου seed για την γεννήτρια τυχαίων αριθμών παράγει την ίδια ακολουθία.

Στη συνέχεια παράγεται τυχαίο σημείο εκκίνησης εντός του search space [-2,2] για την τρέχουσα δοκιμή. Η εντολή **drand48()** παράγει έναν τυχαίο αριθμό κινητής υποδιαστολής ομοιόμορφα κατανεμημένο στην περιοχή [0.0, 1.0].

Ο τύπος $\text{lower}[i] + (\text{upper}[i] - \text{lower}[i]) * \text{drand48}()$ προσαρμόζει τον τυχαίο αριθμό στο επιθυμητό εύρος [lower[i], upper[i]]. Εφόσον στην συγκεκριμένη περίπτωση έχουμε nvars ίσο του 4 για αυτό παράγουμε ένα σημείο με 4 συντεταγμένες αφού ο simplex έχει 4 διαστάσεις.

Στη συνέχεια μέσω της συνάρτησης **initialize_simplex()** δημιουργούνται και τα υπόλοιπα σημεία.

For nvars = 4

Initial Point: startpt = [a, b, c, d].

Simplex Vertices:

- Vertex 0: [a, b, c, d]
- Vertex 1: [a + delta, b, c, d]
- Vertex 2: [a, b + delta, c, d]
- Vertex 3: [a, b, c + delta, d]
- Vertex 4: [a, b, c, d + delta]

Ερώτημα 1 : OpenMP

funevals

Παρατηρούμε ότι στην συνάρτηση f υπάρχει ο μετρητής funevals στον οποίο προσθέτουμε την εντολή `# pragma omp atomic`. Η funevals όπως αναφέραμε είναι μια global μεταβλητή η οποία μετρά τον συνολικό αριθμό υπολογισμού της Evaluation Function. Έτσι κάθε thread θα αυξήσει την συγκεκριμένη μεταβλητή κάθε φορά που καλείται η συνάρτηση f.

Αν δεν υπάρχει κάποιος συγχρονισμός τότε θα δημιουργηθεί race condition καθώς πολλά threads μπορεί να επιχειρήσουν να αυξήσουν τα funevals ταυτόχρονα. Εδώ λοιπόν μπορεί να χάσουμε αυξήσεις. Αν για παράδειγμα η μεταβλητή έχει την τιμή 10 και 2 threads επιχειρήσουν ταυτόχρονα να την αυξήσουν, είναι πιθανό να χαθεί η μία αύξηση και γενικότερα στον συνολικό μετρητή εν τέλη να δούμε αισθητά μειωμένη τιμή. Επίσης αυτό συνεπάγεται ότι τη δεδομένη στιγμή που κάποιο thread επιχειρήσει να αυξήσει την μεταβλητή, η τιμή της να είναι ασυνεπής.

omp_get_time()

Παρατηρούμε ότι αντικαταστήσαμε την συνάρτηση μέτρησης του χρόνου η οποία χρησιμοποιεί την gettimeofday() με την εντολή omp_get_time(). Τα πλεονεκτήματα είναι τα εξής :

```
18  /* Rosenbrock classic parabolic valley ("banana") function */
19
20  double f(double *x, int n)
21  {
22      double fv;
23      int i;
24
25      #pragma omp atomic
26      funevals++;
27
28      fv = 0.0;
29      for (i = 0; i < n - 1; i++){ /* rosenbrock */
30          fv = fv + 100.0 * pow((x[i + 1] - x[i] * x[i]), 2) + pow((x[i] - 1.0), 2);
31      }
32
33      usleep(10);
34      return fv;
35  }
```

- Η omp_get_wtime() προσφέρει μεγαλύτερη ακρίβεια και υψηλότερη ανάλυση σε σχέση με την gettimeofday() γεγονός πολύ σημαντικό στην παράλληλη επεξεργασία.
- Η omp_get_time έχει σχεδιαστεί ειδικά για την μέτρηση της απόδοσης σε παράλληλα προγράμματα και εξασφαλίζει συνεπή μέτρηση με τη χρήση διαφορετικών threads. Αντίθετα η gettimeofday() μπορεί να εμφανίσει ασυνέπειες ρολογιού. Έτσι θα λέγαμε ότι η συγκεκριμένα συνάρτηση είναι thread safe και συνιστάται στην παράλληλη βελτιστοποίηση.

```
77
78  /* initialization of lower and upper bounds of search space */
79
80  for (i = 0; i < MAXVARS; i++) lower[i] = -2.0; /* lower bound: -2.0 */
81  for (i = 0; i < MAXVARS; i++) upper[i] = +2.0; /* upper bound: +2.0 */
82
83  long tseed = 1; // Fixed seed for reproducibility
84
85  t0 = omp_get_wtime();
86
```

#pragma omp parallel reduction(+:funevals)

Στη συνέχεια με την παραπάνω εντολή δημιουργούμε την παράλληλη περιοχή όπου τα threads θα εκτελέσουν τον κώδικα ο οποίος υπάρχει μέσα σε αυτό το block.

Επιπλέον με την εντολή του reduction υποδεικνύουμε στο OpenMP να δημιουργήσει είναι private αντίγραφο των funevals σε κάθε νήμα. Στο τέλος της εκτέλεσης αυτά τα αντίγραφα συνδυάζονται και μάλιστα προστίθενται (+ :) για να αναπαριστούμε με ακρίβεια το συνολικό αριθμό της μεταβλητής funevals.

```
86
87 // #pragma omp parallel reduction(+:funevals) private(trial, i, startpt, endpt, fx, nt, nf)
88 {
89     unsigned short randBuffer[3];
90     randBuffer[0] = 0;
91     randBuffer[1] = 0;
92     randBuffer[2] = tseed + ntrials + omp_get_thread_num(); // Ensure unique seed for each thread
93     //printf("Startpt %d: |", omp_get_thread_num());
94 }
```

Private Variables

Το private clause υποδηλώνει ότι οι συγκεκριμένες μεταβλητές θα πρέπει να είναι ιδιωτικές ανά νήμα και το καθένα να έχει το δικό του αντίγραφο αυτών των variables. Με αυτό τον τρόπο θα αποτρέψουμε race conditions των μεταβλητών αυτών.

- **trial**: κάθε νήμα πρέπει να έχει το δικό του trial για να μην έχουμε race conditions.
- **i**: δείκτης loop for που χρησιμοποιείται εντός παράλληλης περιοχής.
- **startpt**: κάθε thread θα πρέπει να έχει δικό του σημείο έναρξης.
- **endpt**: κάθε thread θα πρέπει να έχει δικό του σημείο λήξης.
- **fx**: τιμή της συνάρτησης Rosenbrock στο τελικό σημείο του MDS.
- **nt, nf**: counters επαναλήψεων και evaluation functions που χρησιμοποιούνται από τον MDS.

Αν αυτές οι μεταβλητές δεν ήταν private τότε τα threads θα τις μοιράζονταν, θα είχαμε λοιπόν data races καθώς τα threads θα αντικαθιστούσαν τιμές των υπολοίπων με αποτέλεσμα λανθασμένα αποτελέσματα.

randBuffer

Ο πίνακας randBuffer χρησιμοποιείται για την αποθήκευση του seed για την συνάρτηση erand48(). Ο αρχικοποιείται με 3 ακεραίους χωρίς πρόσημο. Πιο συγκεκριμένα, τα randBuffer[0] και randBuffer[1] αρχικοποιούνται με την τιμή 0 για λόγους απλότητας ενώ ο randBuffer[2] τίθεται σε μία μοναδική τιμή για κάθε thread προσθέτοντας τις μεταβλητές tseed + ntrials + omp_get_thread_num().

Το `ntrials` το οποίο είναι οι επαναλήψεις του προβλήματος είναι 64 και προσθέτει μεγαλύτερη μεταβλητότητα ενώ το `omp_get_thread_num()` εξασφαλίζει ότι το κάθε νήμα θα κάνει διαφορετικό seeding.

```
86
87
88     #pragma omp parallel reduction(+:funvals) private(trial, i, startpt, endpt, fx, nt, nf)
89     {
90         unsigned short randBuffer[3];
91         randBuffer[0] = 0;
92         randBuffer[1] = 0;
93         randBuffer[2] = tseed + ntrials + omp_get_thread_num(); // Ensure unique seed for each thread
94         //printf("Startpt {%d}: ||", omp_get_thread_num());
95
96         #pragma omp for schedule(dynamic)
97         for (trial = 0; trial < ntrials; trial++) {
98             /* starting guess for rosenbrock test function, search space in [-2, 2) */
99             for (i = 0; i < nvars; i++) {
100                 startpt[i] = lower[i] + (upper[i] - lower[i]) * erand48(randBuffer);
101             }
```

Στον παραπάνω κώδικα παρατηρούμε ότι κάθε νήμα χρησιμοποιεί το δικό του `randBuffer` για τη δημιουργία τυχαίων αριθμών με το `erand48()`, διασφαλίζοντας ότι δεν υπάρχουν παρεμβολές μεταξύ των threads. Επιπλέον κάθε thread παράγει μοναδικό σημείο εκκίνησης για το πρόβλημα βελτιστοποίησης το οποίο πρέπει να λυθεί.

Σημασία χρήσης κατάλληλου seeding

Το seeding το οποίο θα ακολουθήσουμε είναι ζωτικής σημασίας καθώς εάν τα seeds δεν διαφοροποιούνται επαρκώς τότε θα λάβουμε μεγαλύτερο `Fx` δηλαδή δεν θα έχουμε αποτελεσματικότητα στη βελτιστοποίηση `multistart`. Επιπρόσθετα για παράδειγμα δεν χρησιμοποιήσαμε seeding της μορφής `time(NULL)` με σκοπό να εξασφαλίσουμε το `reproducibility`. Πιο συγκεκριμένα θέλαμε να αποφύγουμε την τυχειότητα καθώς με την χρήση της `time(NULL)` στο seeding παρατηρούσαμε μεγάλες μεταβολές της συνάρτησης και της εύρεσης τοπικού ελαχίστου από εκτέλεση σε εκτέλεση.

Scheduling

Η αλήθεια είναι ότι μεταξύ χρήσης `dynamic` ή `static scheduling` δεν παρατηρήθηκαν διαφορές στον χρόνο εκτέλεσης. Αυτό σημαίνει ότι ο φόρτος εργασίας τον οποίο αναλαμβάνει το κάθε thread είναι ο ίδιος και σημαίνει ότι το πρόβλημα μπορεί να παραλληλοποιηθεί αποτελεσματικά χωρίς να απαιτείται ρύθμιση της στρατηγικής που θα ακολουθηθεί στον χρονοπρογραμματισμό.

Ωστόσο επιλέξαμε τον `dynamic` καθώς εξ ορισμού αν ένα thread ολοκληρώσει το chunk του τότε δυναμικά αναλαμβάνει ένα άλλο chunk μέχρι να ολοκληρωθούν οι επαναλήψεις. Έτσι διασφαλίζουμε καλύτερη εξισορρόπηση φορτίου και επίσης είναι πιο αποδοτικό αν κάποιες επαναλήψεις διαρκέσουν λιγότερο από άλλες δηλαδή αν ο MDS ο οποίος καλείται είναι non – uniform σαν συνάρτηση.

Critical Section

Το critical section χρησιμοποιείται για να διασφαλίσουμε ότι κάθε φορά ένα μόνο νήμα θα ενημερώνει τις μεταβλητές `best_fx`, `best_trial`, `best_nt`, `best_nf` και `best_pt`. Αυτές οι μεταβλητές περιέχουν την καλύτερη λύση που έχει βρεθεί μέχρι στιγμής και το να επιτρέπεται σε πολλά νήματα να τις ενημερώνουν ταυτόχρονα θα μπορούσε να οδηγήσει σε λανθασμένα αποτελέσματα. Όταν πολλά threads προσπαθήσουν να ενημερώσουν τις συγκεκριμένες μεταβλητές θα δημιουργηθούν race conditions. Ωστόσο θα πρέπει να σημειώσουμε ότι τα critical sections εφόσον εκτελούνται από ένα thread τη φορά επιβαρύνουν τον συνολικό χρόνο εκτέλεσης.

```
106         #pragma omp critical
107         {
108             /* keep the best solution */
109             if (fx < best_fx) {
110                 best_trial = trial;
111                 best_nt = nt;
112                 best_nf = nf;
113                 best_fx = fx;
114                 for (i = 0; i < nvars; i++)
115                     best_pt[i] = endpt[i];
116             }
117         }
118     }
119 }
120
```

Ερώτημα 2 : OpenMP Tasks

Ανάλυση

Με τη χρήση OpenMP Tasks μας δίνεται επιπλέον ευελιξία σχετικά με τη παραλληλοποίηση το προβλήματος. Κάθε task δημιουργεί overhead για τη δημιουργία, διαχείριση και ολοκλήρωση του, συνεπώς με τη χρήση πολλών task το overhead μπορεί να ξεπεράσει τα οφέλη της παράλληλης εκτέλεσης. Για την αποφυγή του παραπάνω προβλήματος χρησιμοποιήθηκε το εργαλείο 'gprof' για την χρονική ανάλυση το σειριακού κώδικα.

```
Flat profile:
Each sample counts as 0.01 seconds.
```

% time	cumulative seconds	self seconds	calls	self us/call	total us/call	name
100.00	0.01	0.01	640292	0.02	0.02	f
0.00	0.01	0.00	80149	0.00	0.00	swap_simplex
0.00	0.01	0.00	80000	0.00	0.00	simplex_size
0.00	0.01	0.00	64	0.00	156.25	mds

Το εργαλείο μας δείχνει πως η συνάρτηση Rosenbrock, δηλαδή η συνάρτηση f, καταναλώνει το μεγαλύτερο μέρος του χρόνου που απαιτείται για την εκτέλεση του προγράμματος. Σύμφωνα με τα παραπάνω μπορούμε να αποφανθούμε πως η παραλληλοποίηση θα γίνει στην συνάρτηση f.

Υλοποίηση

Η μεθοδολογία του OpenMp Tasks είναι παρόμοια με το πρώτο ερώτημα σχετικά με την αρχικοποίηση και το seeding οπότε θα αναλυθούν οι διαφορές τους.

```
88      #pragma omp parallel
89      {
90          #pragma omp single
91          {
92
93              unsigned short randBuffer[3];
94
95              //int thread_id = omp_get_thread_num();
96
97              randBuffer[0] = 0;
98              randBuffer[1] = 0;
99              randBuffer[2] = tseed + ntrials + 1; // Ensure unique seed for each thread
100
101              //printf("\n\nStart Trials ...");
```

#pragma omp parallel

Δημιουργεί μια ομάδα threads για παράλληλη εκτέλεση (ορίζει την παράλληλη περιοχή).

#pragma omp single

Σύμφωνα με τη θεωρία για τη χρήση του OpenMP tasks πρέπει η mds να εκτελεστεί από ένα νήμα και αυτό μας το προσφέρει η "omp single". Με αυτό το τρόπο τα υπόλοιπα νήματα μένουν 'ελεύθερα' έως ότου να καλεστούν από ένα task.

Επαναληπτική δομή για τον mds

Από τη στιγμή που η επαναληπτική δομή περικλείεται από τη εντολή που αναλύθηκε προηγούμενος ("omp single") και εκτελείται από ένα νήμα δεν χρειάζεται κάποια τροποποίηση από τον σειριακό. Για παράδειγμα η 'omp critical' που χρησιμοποιήθηκε πριν για τον υπολογισμό των best μεταβλητών αφαιρέθηκε μιάς και δεν θα προσπελαστεί απο παραπάνω από ένα νήμα.

```
1  for (trial = 0; trial < ntrials; trial++) {
2
3      //printf("\n\nThread num %d || Rand Buffer: %d",omp_get_thread_num(),randBuffer[2]);
4
5      /* starting guess for rosenbrock test function, search space in [-2, 2) */
6      for (i = 0; i < nvars; i++) {
7          startpt[i] = lower[i] + (upper[i] - lower[i]) * erand48(randBuffer);
8      }
9
10     int term = -1;
11
12     mds(startpt, endpt, nvars, &fx, eps, maxfevals, maxiter, mu, theta, delta, &nt, &nf, lower, upper, &term);
13
14
15     /* keep the best solution */
16     if (fx < best_fx) {
17         best_trial = trial;
18         best_nt = nt;
19         best_nf = nf;
20         best_fx = fx;
21         for (i = 0; i < nvars; i++)
22             best_pt[i] = endpt[i];
23     }
24
25
26 }
```

#pragma omp task

Τέλος μπορούμε να δημιουργούμε tasks στα σημεία του κώδικα που το χρειάζονται. Στη περίπτωση μας καταλήξαμε πως παραλληλοποίηση χρειάζεται η Rosenbrock οπότε ορίσαμε tasks για κάθε κάλεσμα της στον mds.

```

1  for (i = 1; i < n + 1; i++) {
2      for (j = 0; j < n; j++) {
3          ec[i * n + j] = u[0 * n + j] + theta * ((u[i * n + j]
4              - u[0 * n + j]));
5      }
6
7      #pragma omp task firstprivate(i) untied
8      fec[i] = f(&ec[i * n], n);
9      *nf = *nf + 1;
10 }
11
12 #pragma omp taskwait

```

Παραπάνω βλέπουμε μία εφαρμογή του `task`. Η `'omp task'` δημιουργεί μια μονάδα εργασίας που μπορεί να εκτελεστεί από οποιοδήποτε thread στην ομάδα OpenMp. Η `firstprivate(i)` ορίζει την ιδιωτικότητα του (i) για κάθε thread. Το `untied` clause καθορίζει ότι το task μπορεί να το αναλάβει και να το εκτελέσει οποιοδήποτε διαθέσιμο thread ανεξάρτητα από το πιο thread το δημιούργησε. Ομοίως υλοποιήθηκαν και τα υπόλοιπα σημεία που καλείται η Rosenbrock.

#pragma omp taskwait

Η συγκεκριμένη εντολή μας εξασφαλίζει την ολοκλήρωση εκτέλεσης του `task`. Από την στιγμή που η `'fec'` θα ξαναχρησιμοποιήσει πρέπει να εξασφαλιστεί πως η Rosenbrock έχει επιστρέψει πρώτου συνεχίσει η εκτέλεση. Βέβαια η `'fec'` θα χρησιμοποιηθεί μετά την ολοκλήρωση της αρχικής επαναληπτικής δομής `for` γι' αυτό το `taskwait` μπήκε μετά. Έτσι δημιουργούνται task για κάθε `i` από 1 έως `n-1` και αφού χρησιμοποιούν διαφορετικά κομμάτια του πίνακα `'ec'` δεν έχουμε race conditions. Ειδικότερα, η εμφωλευμένη `for` θα υπολογίσει το κομμάτι του πίνακα `'ec'` το οποίο θα χρησιμοποιήσει η Rosenbrock στο κάθε `i`. Μόλις ολοκληρωθεί αυτός ο υπολογισμός ορίζεται task για να υπολογίσει το `fec[i]` χρησιμοποιώντας το κομμάτι του πίνακα `'ec'` που μόλις υπολογίστηκε. Έπειτα, δεν χρειάζεται να περιμένει να ολοκληρωθεί το task οπότε προχωράει στο επόμενο `'i'`.

Rosenbrock

```
1 double f(double *x, int n)
2 {
3     double fv;
4     int i;
5
6     #pragma omp atomic
7     funevals++;
8
9     fv = 0.0;
10
11     for (i = 0; i < n - 1; i++){ /* rosenbrock */
12         fv = fv + 100.0 * pow((x[i + 1] - x[i] * x[i]), 2) + pow((x[i] - 1.0), 2);
13     }
14
15     #pragma omp task untied
16     usleep(1); /* do not remove, introduces some artificial work */
17
18     return fv;
19 }
20
```

Μέσα στην συνάρτηση καλείται η `usleep` ή οποία έχει σκοπό να προσθέσει ένα τεχνητό φόρτο εργασίας. Μπορούμε να την συσχετίσουμε με ένα φόρτο το οποίο δεν σχετίζεται με τα προηγούμενα. Πιο συγκεκριμένα, αν υποθέσουμε πως δεν επηρεάζει τον υπολογισμό του `fv` μπορούμε να της ορίσουμε ένα δικό της task στο οποίο θα δώσουμε και εδώ το clause `untied`. Επομένως, η συνάρτηση `f` επιστρέψει το `fv` χωρίς να περιμένει την ολοκλήρωση του `usleep`.

Ερώτημα 3 : MPI

MPI enviroment

Στο συγκεκριμένο ερώτημα θα χρησιμοποιήσουμε MPI το οποίο αποτελεί ένα σύστημα μηνυμάτων σε αρχιτεκτονικές παράλληλων υπολογιστών. Συγκεκριμένα επιτρέπει σε πολλαπλές διεργασίες να επικοινωνούν μεταξύ τους αποστέλλοντας και λαμβάνοντας μηνύματα. Το MPI λειτουργεί με την αρχικοποίηση πολλαπλών διεργασιών (processes) σε ένα σύστημα για παράδειγμα όπως και στην προκειμένη περίπτωση με πολλούς πυρήνες δηλαδή σε multi-core συστήματα.

MPI_Init(&argc, &argv): αρχικοποιεί το MPI περιβάλλον.

MPI_Comm_rank(MPI_COMM_WORLD, &rank): η συγκεκριμένη εντολή χρησιμοποιείται για να καθορίσει το rank για την κάθε διεργασία στον communicator. Στην προκειμένη περίπτωση communicator είναι ο MPI_COMM_WORLD. Αν για παράδειγμα τρέξουμε το executable με 8 processes τότε θα έχουμε ranks 0, 1, 2, 3, 4, 5, 6 και 7.

MPI_Comm_size(MPI_COMM_WORLD, &size): Καθορίζει το συνολικό αριθμό διεργασιών στον communicator MPI_COMM_WORLD. Στη συγκεκριμένη περίπτωση θα έχουμε size 8.

MPI_Finalize(): τερματίζει το MPI περιβάλλον.

```
37 int main(int argc, char *argv[]) {
38
39     MPI_Init(&argc, &argv);
40
41     int rank, size;
42     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
43     MPI_Comm_size(MPI_COMM_WORLD, &size);
44 }
```

Work Distribution

```
85 /* Calculate work distribution */
86
87 double const step = (double)ntrials / size;
88 unsigned long start = rank * step;
89 unsigned long end = (rank + 1) * step;
90 if (rank == size - 1) end = ntrials;
91
92 for (trial = start; trial < end; trial++) {
93     /* starting guess for rosenbrock test function, search space in [-2, 2) */
94     for (i = 0; i < nvars; i++) {
95         startpt[i] = lower[i] + (upper[i] - lower[i]) * erand48(randBuffer);
96     }
97 }
```

- Ο συνολικός αριθμός των επαναλήψεων (ntrials) διαιρείται με τον αριθμό των διεργασιών (size) για να καθοριστεί πόσα trials θα διαχειριστεί η κάθε διεργασία (process).
- Το **step** αντιπροσωπεύει τον αριθμό των trials όπου θα αναλάβει το κάθε process.
- Η μεταβλητή **start** είναι ο δείκτης για το αρχικό trial του κάθε process.
- Η μεταβλητή **end** είναι ο δείκτης για το τελευταίο trial του κάθε process.
- Η τελευταία διεργασία (rank size - 1) εξασφαλίζει ότι καλύπτει τυχόν υπολειπόμενες δοκιμές εάν το ntrials δεν είναι απόλυτα διαιρετό με το size.

Example

If ntrials = 64 and size = 8

step = $64 / 8 = 8$

- **Process 0 (rank 0):**
 - $start = 0 * 8 = 0$
 - $end = 1 * 8 = 8$
 - This process handles trials 0 to 7.
- **Process 1 (rank 1):**
 - $start = 1 * 8 = 8$
 - $end = 2 * 8 = 16$
 - This process handles trials 8 to 15.
- **Process 7 (rank 7):**
 - $start = 7 * 8 = 56$
 - $end = 8 * 8 = 64$
 - This process handles trials 56 to 63.

Αξίζει να αναφέρουμε ότι στον συγκεκριμένο κώδικα το process με rank 0 συμμετέχει κανονικά στους υπολογισμούς. Εφόσον ολοκληρώσει λοιπόν το rank 0 συγκεντρώνει τα αποτελέσματα από όλες τις διεργασίες και καθορίζει τον συνολικό best result.

Έτσι λοιπόν πετυχαίνουμε αξιοποίηση όλων των διαθέσιμων πόρων που οδηγεί σε μείωση του συνολικού χρόνου υπολογισμού. Τέλος αποφεύγουμε την αδράνεια του rank 0 από το να συλλέγει απλά τα αποτελέσματα χρησιμοποιώντας έτσι αποδοτικά την διαθέσιμη υπολογιστική ισχύς.

Το παρακάτω block κώδικα εκτελείται από κάθε process και ενημερώνει το τοπικά καλύτερο αποτέλεσμα της εάν βρεθεί καλύτερο :

```
97
98     for (trial = start; trial < end; trial++) {
99         /* starting guess for rosenbrock test function, search space in [-2, 2] */
100         for (i = 0; i < nvars; i++) {
101             startpt[i] = lower[i] + (upper[i] - lower[i]) * erand48(randBuffer);
102         }
103
104         int term = -1;
105         mds(startpt, endpt, nvars, &fx, eps, maxfevals, maxiter, mu, theta, delta, &nt, &nf, lower, upper, &term);
106
107         if (fx < local_best_fx) {
108             local_best_trial = trial;
109             local_best_nt = nt;
110             local_best_nf = nf;
111             local_best_fx = fx;
112             for (i = 0; i < nvars; i++)
113                 local_best_pt[i] = endpt[i];
114         }
115     }
116 }
```

Για να διαχωρίσουμε καλύτερα τα local με τα global αποτελέσματα ορίζουμε επιπλέον και διαχωρίζουμε global από local μεταβλητές :

```
62     /* information about the best point found by each process */
63
64     double local_best_pt[MAXVARS];
65     double local_best_fx = 1e10;
66     int local_best_trial = -1;
67     int local_best_nt = -1;
68     int local_best_nf = -1;
69
70     /* global best information */
71
72     double global_best_pt[MAXVARS];
73     double global_best_fx = 1e10;
74     int global_best_trial = -1;
75     int global_best_nt = -1;
76     int global_best_nf = -1;
```

Στην συνέχεια παρατηρούμε το παρακάτω block κώδικα το οποίο διαχωρίζει τις αρμοδιότητες του master rank = 0 και των υπολοίπων. Παρατηρούμε ότι αρχικοποιούνται οι global μεταβλητές με τις local μεταβλητές οι οποίες βρέθηκαν στο rank 0. Αυτό είναι σημαντικό βήμα καθώς θα πρέπει να συμπεριλάβουμε και τα αποτελέσματα του master process :

```
/* Rank 0 also considers its own results */
if (rank == 0) {

    global_best_fx = local_best_fx;
    global_best_trial = local_best_trial;
    global_best_nt = local_best_nt;
    global_best_nf = local_best_nf;

    for (i = 0; i < nvars; i++)
        global_best_pt[i] = local_best_pt[i];
}
```

Στη συνέχεια το rank 0 κάνει receive τα local results από όλα τα υπόλοιπα processes και ενημερώνει αντίστοιχα τις global μεταβλητές, υπολογίζοντας το καλύτερο αποτέλεσμα.

Συγκεκριμένα το rank 0 λαμβάνει πληροφορίες σχετικά με την συνάρτηση fx , το trial στο οποίο βρέθηκε η βέλτιστη fx καθώς και τον αριθμό των επαναλήψεων nt και nf . Τέλος γίνονται receive και πληροφορίες σχετικά με τις συντεταγμένες του βέλτιστου σημείου pt .

```
129 |
130 |     for (int p = 1; p < size; p++) {
131 |         double recv_fx;
132 |         int recv_trial, recv_nt, recv_nf;
133 |         double recv_pt[MAXVARS];
134 |
135 |         MPI_Status status;
136 |
137 |         MPI_Recv(&recv_fx, 1, MPI_DOUBLE, p, 100, MPI_COMM_WORLD, &status);
138 |         MPI_Recv(&recv_trial, 1, MPI_INT, p, 101, MPI_COMM_WORLD, &status);
139 |         MPI_Recv(&recv_nt, 1, MPI_INT, p, 102, MPI_COMM_WORLD, &status);
140 |         MPI_Recv(&recv_nf, 1, MPI_INT, p, 103, MPI_COMM_WORLD, &status);
141 |         MPI_Recv(recv_pt, MAXVARS, MPI_DOUBLE, p, 104, MPI_COMM_WORLD, &status);
142 |
143 |         if (recv_fx < global_best_fx) {
144 |             global_best_fx = recv_fx;
145 |             global_best_trial = recv_trial;
146 |             global_best_nt = recv_nt;
147 |             global_best_nf = recv_nf;
148 |             for (i = 0; i < nvars; i++)
149 |                 global_best_pt[i] = recv_pt[i];
150 |         }
151 |     }
```

Rank 0

`MPI_Recv(&recv_fx, 1, MPI_DOUBLE, p, 100, MPI_COMM_WORLD, &status)`

- **recv_fx**: Η μεταβλητή στην οποία θα αποθηκευτούν τα δεδομένα της fx .
- **1**: Αριθμός των στοιχείων που πρέπει να ληφθούν.
- **MPI_DOUBLE**: Τύπος δεδομένων των στοιχείων.
- **p**: Rank του process το οποίο έστειλε το μήνυμα.
- **100**: tag μηνύματος για την αναγνώριση του μηνύματος.
- **MPI_COMM_WORLD**: Communicator.
- **&status**: μεταβλητή κατάστασης σχετικά με το ληφθέν μήνυμα.

Αντίστοιχα τα υπόλοιπα processes αποστέλλουν τις πληροφορίες στο rank 0. Επίσης έχουμε ορίσει αντίστοιχα το send tag ανάλογα με την μεταβλητή σε 100,101,102 κ.ο.κ. τα οποία αντιστοιχίζονται με τις Receive εντολές :

```
151     }
152
153     } else {
154         MPI_Send(&local_best_fx, 1, MPI_DOUBLE, 0, 100, MPI_COMM_WORLD);
155         MPI_Send(&local_best_trial, 1, MPI_INT, 0, 101, MPI_COMM_WORLD);
156         MPI_Send(&local_best_nt, 1, MPI_INT, 0, 102, MPI_COMM_WORLD);
157         MPI_Send(&local_best_nf, 1, MPI_INT, 0, 103, MPI_COMM_WORLD);
158         MPI_Send(&local_best_pt, MAXVARS, MPI_DOUBLE, 0, 104, MPI_COMM_WORLD);
159     }
160 }
```

MPI_Reduce

```
MPI_Reduce(&local_funevals, &global_funevals, 1, MPI_UNSIGNED_LONG,
MPI_SUM, 0, MPI_COMM_WORLD)
```

Συνδυάζει τα local_funevals από όλες τις διεργασίες σε global_funevals χρησιμοποιώντας την πράξη αθροίσματος.

- **local_funevals:** buffer εισόδου (δεδομένα προς reduction).
- **global_funevals:** buffer εξόδου (αποτέλεσμα της reduction).
- **1:** Αριθμός στοιχείων στον buffer εισόδου.
- **MPI_UNSIGNED_LONG:** Τύπος δεδομένων των στοιχείων.
- **MPI_SUM:** Πράξη προς εκτέλεση (άθροισμα).
- **0:** rank της διεργασίας που θα λάβει το αποτέλεσμα.
- **MPI_COMM_WORLD:** Communicator.

```
161  /* Reduce the function evaluations across all processes */
162  unsigned long global_funevals;
163  MPI_Reduce(&local_funevals, &global_funevals, 1, MPI_UNSIGNED_LONG, MPI_SUM, 0, MPI_COMM_WORLD);
164
165  t1 = get_wtime();
166
167  if (rank == 0) {
168      printf("\n\nFINAL RESULTS (MPI):\n");
169      printf("Elapsed time = %.3lf s\n", t1 - t0);
170      printf("Total number of trials = %d\n", ntrials);
171      printf("Total number of function evaluations = %ld\n", global_funevals);
172      printf("Best result at trial %d used %d iterations, %d function calls and returned\n", global_best_trial, global_best_nt, global_best_nf);
173      for (i = 0; i < nvars; i++) {
174          printf("x[%3d] = %15.7le\n", i, global_best_pt[i]);
175      }
176      printf("f(x) = %15.7le\n", global_best_fx);
177  }
178
179  MPI_Finalize();
180  return 0;
181
182 }
```

Print Results

Το rank 0 εκτυπώνει τα τελικά αποτελέσματα :

```
166
167     if (rank == 0) {
168         printf("\n\nFINAL RESULTS (MPI):\n");
169         printf("Elapsed time = %.3lf s\n", t1 - t0);
170         printf("Total number of trials = %d\n", ntrials);
171         printf("Total number of function evaluations = %d\n", global_funevals);
172         printf("Best result at trial %d used %d iterations, %d function calls and returned\n", global_best_trial, global_best_nt, global_best_nf);
173         for (i = 0; i < nvars; i++) {
174             printf("x[%3d] = %15.7le \n", i, global_best_pt[i]);
175         }
176         printf("f(x) = %15.7le\n", global_best_fx);
177     }
178
179     MPI_Finalize();
180     return 0;
181 }
```

Αποτελέσματα

Το μηχάνημα στο οποίο έγιναν οι μετρήσεις είναι ένα Macbook Air 2022 το οποίο έχει τον M2 επεξεργαστή με τα παρακάτω χαρακτηριστικά :

Αρχιτεκτονική CPU

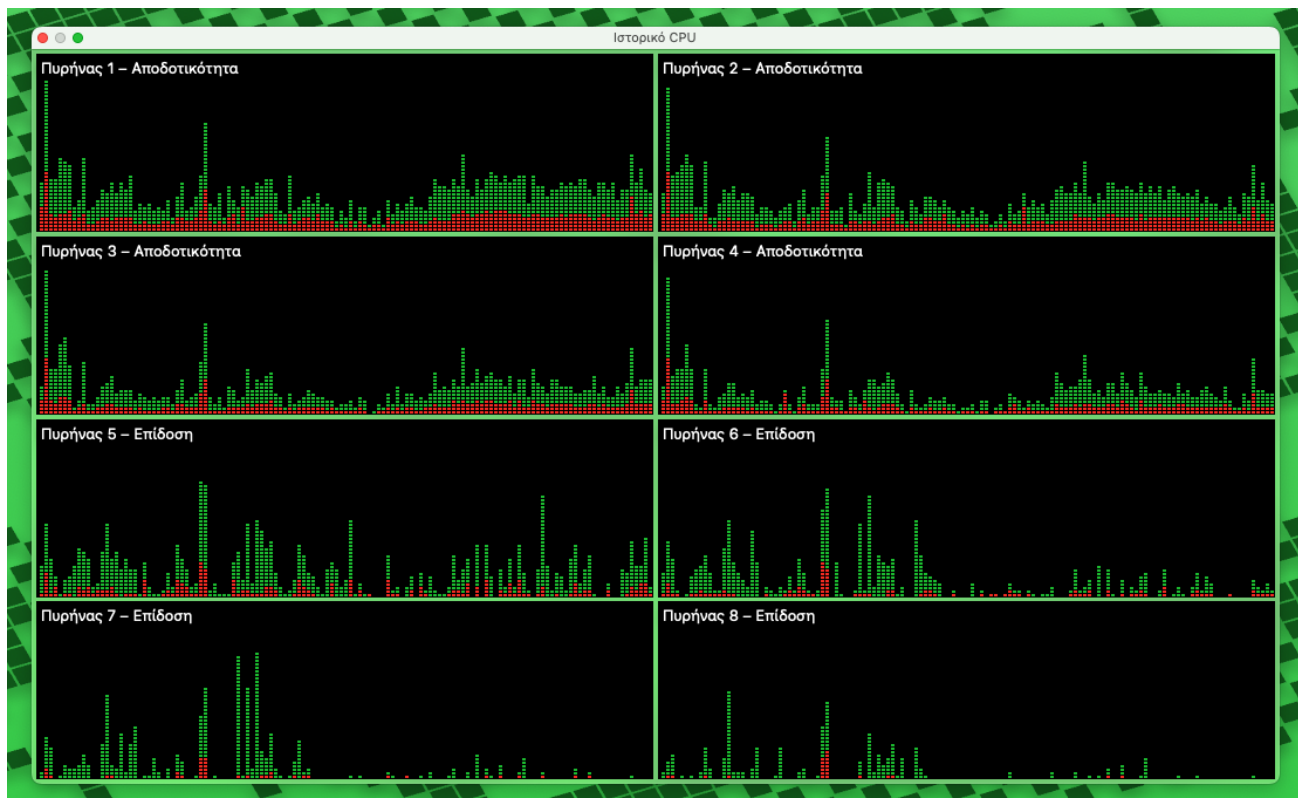
CPU: 8-Core με συνδυασμό τεσσάρων πυρήνων υψηλής απόδοσης (power-cores) και τεσσάρων efficiency cores. Αυτή η αρχιτεκτονική επιτρέπει την αποτελεσματική παράλληλη εκτέλεση εργασιών και την εξισορρόπηση φορτίου μεταξύ των πυρήνων.

system_profiler SPHardwareDataType

```
Hardware:
  Hardware Overview:
    Model Name: MacBook Air
    Model Identifier: Mac14,2
    Model Number: Z15S000W1GR/A
    Chip: Apple M2
    Total Number of Cores: 8 (4 performance and 4 efficiency)
    Memory: 16 GB
```

sysctl -a | grep machdep.cpu

```
machdep.cpu.cores_per_package: 8
machdep.cpu.core_count: 8
machdep.cpu.logical_per_package: 8
machdep.cpu.thread_count: 8
machdep.cpu.brand_string: Apple M2
```



Μετρήσεις

Σημείωση: για να εκτελεστούν τα python scripts (thread_plots.py, plots.py) πρέπει να έχουν εγκατασταθεί οι βιβλιοθήκες pandas, matplotlib, seaborn:

```
pip install pandas matplotlib seaborn
```

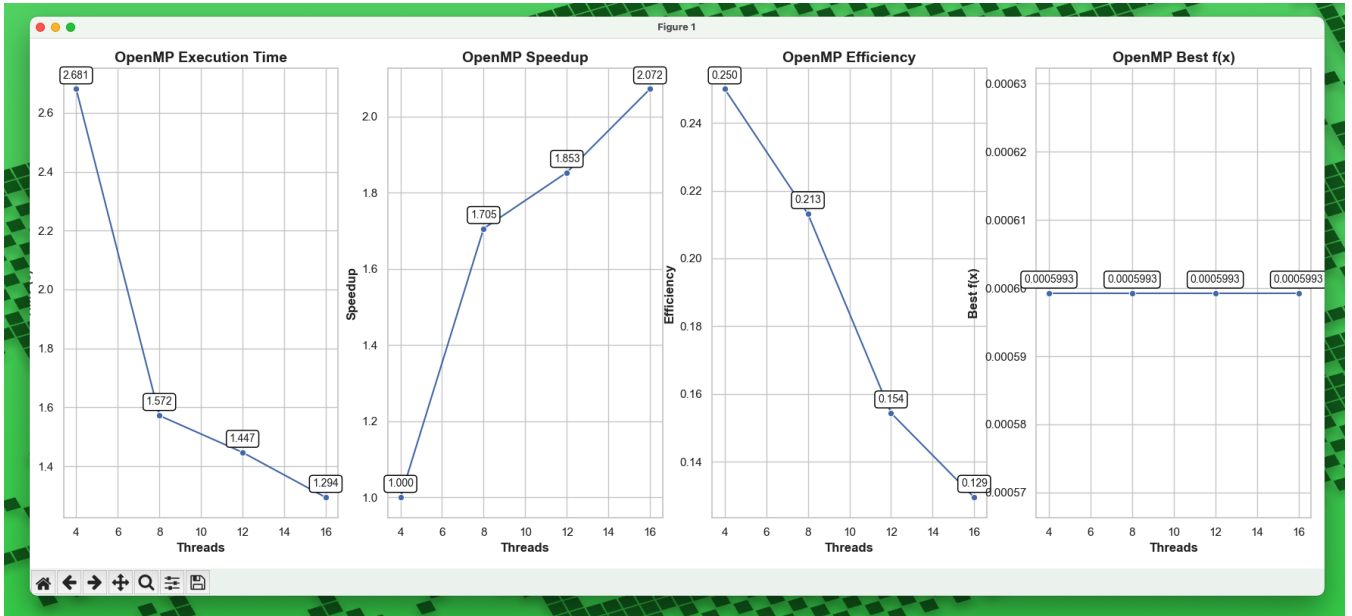
Αρχικά υλοποιήθηκε script στο οποίο δοκιμάζουμε την εκτέλεση της παραλληλοποίησης σε 4 ,8 ,12 και 16 threads. Εκτελούμε το python αρχείο thread_plots.py στο οποίο παρατηρούμε ότι πραγματοποιούμε τις παρακάτω εκτελέσεις :

```
11 # Number of threads/processes to test
12 thread_counts = [4, 8, 12, 16]
13
14 # Executable commands templates
15 executables = {
16     'openmp': './multistart_mds_omp {threads}',
17     'openmp_tasks': './multistart_mds_omp_tasks {threads}',
18     'mpi': 'mpirun -n {threads} ./multistart_mds_mpi'
19 }
```

Στη συνέχεια στα αντίστοιχα json files γράφουμε τα αποτελέσματα των εκτελέσεων έτσι ώστε αν thread selections να δούμε που παρατηρείται το μεγαλύτερο speedup.

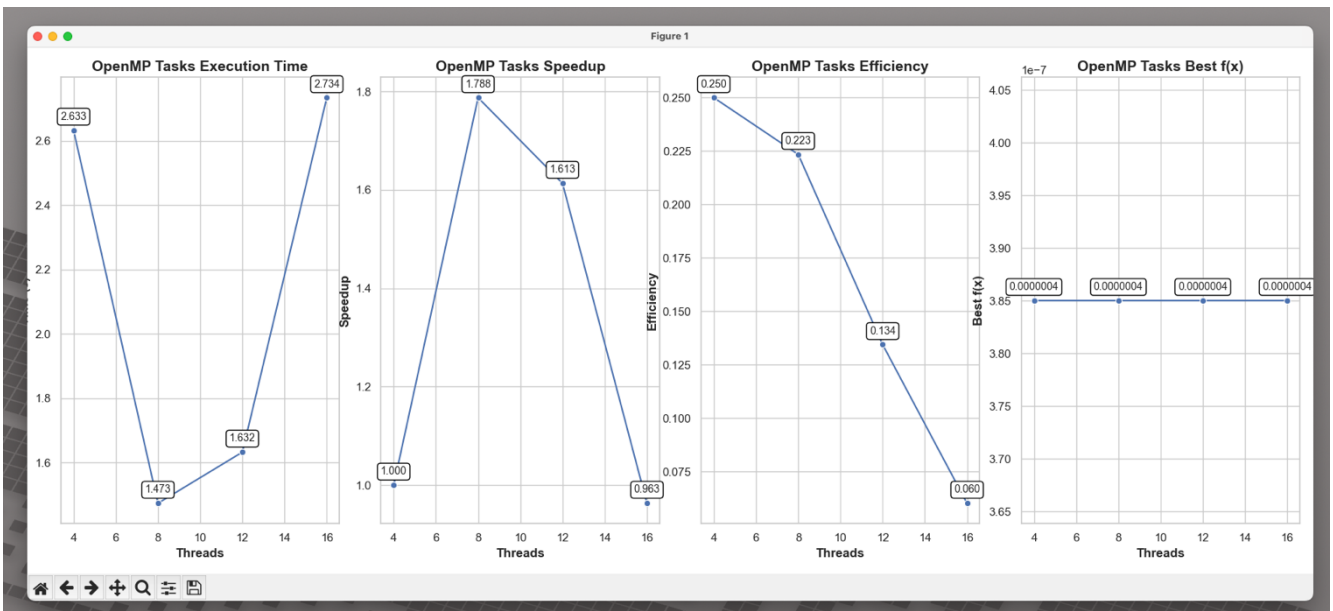
OpenMP

Στο μοντέλο OpenMP παρατηρείται το μεγαλύτερο speedup στα 16 threads παρόλο που στο σύστημα μας έχουμε 8 threads.



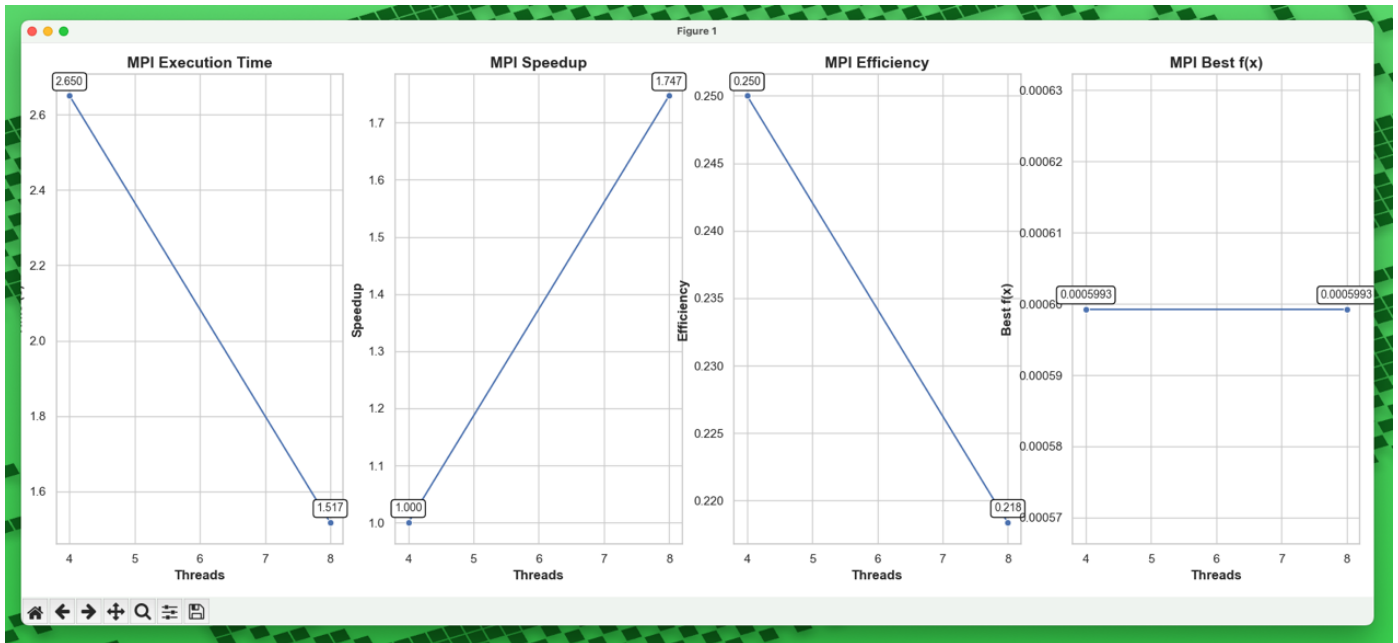
OpenMP Tasks

Παρατηρούμε στα tasks ότι λαμβάνουμε καλύτερους χρόνους χρησιμοποιώντας 8 threads.



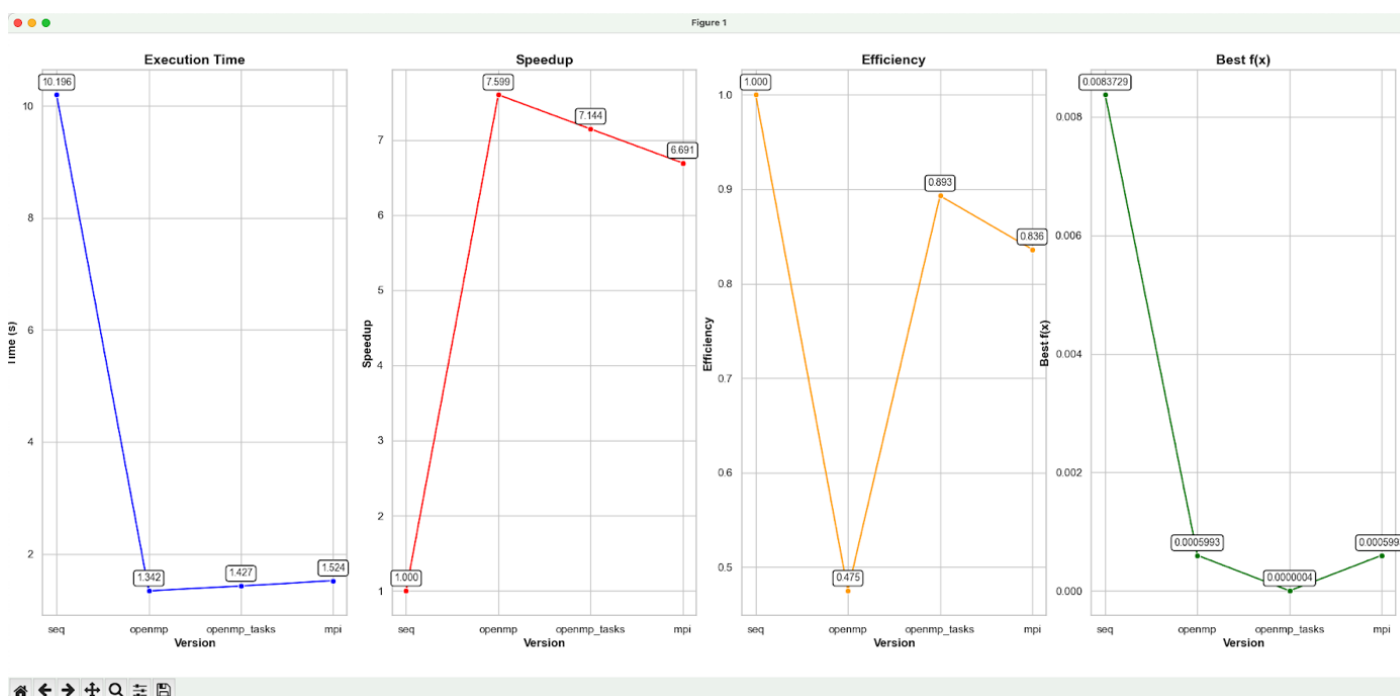
MPI

Εδώ στο σύστημα μας μπορούμε να χρησιμοποιήσουμε έως 8 processes οπότε πραγματοποιούμε τις μετρήσεις για 4 και 8 processes.



Σύγκριση Μεθόδων

Για την σύγκριση μεταξύ των μεθόδων υλοποιήθηκε το script `plots.py` σε `python` με σκοπό την σχεδίαση των διαγραμμάτων. Για να εξασφαλιστούν ορθές τιμές η κάθε υλοποίηση παραλληλοποίησης εκτελείτε πολλαπλές φορές (5) και κρατάει τον μέσο όρο για την σχεδίαση των τιμών για τη δημιουργία των διαγραμμάτων. Για να έχουμε καλύτερα αποτελέσματα το seeding σε κάθε υλοποίηση έχει διαμορφωθεί με αυτο το σκοπό. Το `tseed` και `ntrial` είναι σταθεροί παράγοντες στην εκάστοτε υλοποίηση αλλά στο `openmp` προστίθεται το `thread num` ενώ στην `mpi` προστίθεται το `rank`. Με αυτό τον τρόπο παρατηρήσαμε ότι λαμβάνουμε το καλύτερο δυνατό $f(x)$.



Συμπεράσματα

Αρχικά βλέπουμε πως η παραλληλοποίηση μας επιστρέφει σημαντική βελτίωση στην χρονική απόδοση του συστήματος. Είναι σημαντικό να αναφερθεί πως η παραλληλοποίηση διαφέρει ανάλογα με το σύστημα στο οποίο χρησιμοποιείται (ex. αρχιτεκτονική, single core speed, total threads). Πιο συγκεκριμένα, στη δική μας περίπτωση τον περισσότερο υπολογιστικό χρόνο τον δεσμεύει η usleep οπότε όταν υλοποιήθηκε η παραλληλοποίηση με tasks δόθηκε περισσότερο βάρος εκεί.

Συγκρίνοντας το χρόνο μεταξύ των τριών υλοποιήσεων παραλληλοποίησης βλέπουμε παραπλήσια χρονική απόδοση συνεπώς στο συγκεκριμένο πρόβλημα είναι εφικτή η χρήση και των τριών. Βέβαια OpenMP Tasks χρειάστηκε ανάλυση του κώδικα ώστε να εντοπιστεί το πιο χρονοβόρο σημείο το οποίο μπορεί να μην εφικτό σε μεγαλύτερα προβλήματα ή τουλάχιστον να μην προσφέρει σημαντική βελτίωση σε σχέση με το χρόνο που χρειάζεται για να παραλληλοποιηθούν μεμονωμένα σημεία. Από την άλλη πλευρά, το openmp προσφέρει το καλύτερο χρόνο και σε ένα πρόβλημα μεγαλύτερης κλίμακας αλλά ίδιας φύσης μπορεί η διαφορά μεταξύ των υλοποιήσεων να ήταν μεγαλύτερη και να η σωστή επιλογή να ήταν η χρήση του.

Συγκρίνοντας το $f(x)$ βλέπουμε το καλύτερο αποτέλεσμα στο OpenMP tasks οπότε σε ένα πρόβλημα που το error είναι ο κύριος παράγοντας θα ήταν προτιμότερη χρήση του. Βέβαια παρατηρώντας το efficiency το OpenMP δεν χρησιμοποιεί τους πόρους του συστήματος τόσο αποδοτικά. Δηλαδή η αύξηση του χρόνου που προσφέρει δεν είναι αρκετή για τους πόρους που δεσμεύει. Σε ένα σύστημα που οι πόροι είναι περιορισμένοι ίσως να μην είναι η καλύτερη επιλογή. Είναι σημαντικό να τονιστεί πως η κάθε υλοποίηση δεν έχει διαφορά

στους υπολογισμούς που κάνει, ο λόγος για τον οποίο έχουμε διαφορά στο error είναι οι μικρές τροποποιήσεις στο seeding. Για παράδειγμα, το OpenMP tasks πήρε αρχικές τιμές στο 'startpt' οι οποίες κατάφεραν να έχουν τη σύγκλιση που βλέπουμε. Αν δώσουμε τις ίδιες αρχικές τιμές σε οπουδήποτε άλλη υλοποίηση θα μας επιστρέφει ακριβώς το ίδιο error.

Συνοψίζοντας, κάθε υλοποίηση έχει προτερήματα και μειονεκτήματα το οποίο επηρεάζονται σημαντικά από αρκετούς παράγοντες. Είναι σημαντικό σε κάθε πρόβλημα να εξετάζεται τι θα αποφέρει τα καλύτερα αποτελέσματα ανάλογα με τις απαιτήσεις. Στο δικό μας πρόβλημα και οι 3 λύσεις μας επιστρέφουν πολύ συγκρίσιμα αποτελέσματα και δεν μπορούν να αποφανθούμε πως μια υλοποίηση υπερτερεί μιας άλλης γιατί αυτό εξαρτάται από το ποιος παράγοντας έχει περισσότερη βαρύτητα (execution time, efficiency, best $f(x)$).